

AUFGABE ZUR VORLESUNG

Distributed Parcel System – REST entwerfen und implementieren

Modul	3IT-VSIT-50 · Verteilte Systeme und Internet der Dinge
Thema	Ressourcen, Idempotenz, bedingte Requests, Nebenläufigkeit, Caching
Dozent	Dr. Ulrich Winkler
Semester	5. Semester · Informationstechnik
Bearbeitungszeit	ca. 12–16 Stunden

Vorlesung: Decks *REST* und *Partial Failure*

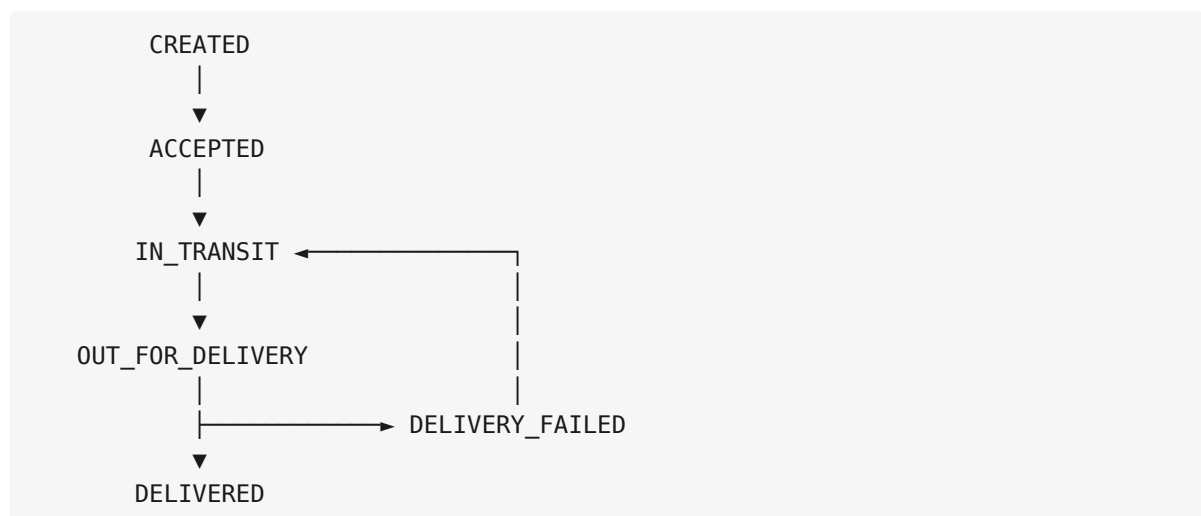
Ziel: Eine REST-API für einen Paketdienst **entwerfen** und in Python **implementieren** – so, dass sie sich auch dann noch definiert verhält, wenn Antworten verloren gehen, Requests doppelt ankommen, zwei Clients gleichzeitig dieselbe Ressource ändern und der Server mitten im Schreiben abstürzt.

Szenario

Sie bauen den Kern eines Paketdienstes, stark vereinfacht: ein Kunde gibt ein Paket auf, es wandert durch das Netz des Dienstleisters, ein Fahrer stellt es zu, jemand bezahlt den Versand. An dieser Domäne hängen alle interessanten REST-Fragen, ohne dass Sie sich mit Fachlogik herumschlagen müssen.

Die Clients Ihrer API sind **keine Browser mit Benutzer davor**, sondern andere Programme: das Handterminal eines Fahrers im Funkloch, ein Sortierautomat, die Webseite der Sendungsverfolgung. Diese Clients machen genau das, was Software in unzuverlässigen Netzen tut – sie wiederholen Requests, wenn keine Antwort kommt. Ihre API muss das aushalten.

Ein Paket durchläuft diese Zustände:



DELIVERED ist ein Endzustand. **DELIVERY_FAILED** ist es nicht: Das Paket geht zurück ins Depot (**IN_TRANSIT**), ein neuer Zustellversuch folgt. Ein Paket, das bereits zugestellt ist, kann nicht wieder

`IN_TRANSIT` werden – ein Request, der das versucht, ist ein Fehler und muss als solcher beantwortet werden.

Keine Datenbank

Ihre Daten liegen in **Dateien** unter `$DATA_DIR`. Das ist Absicht. Mit einer relationalen Datenbank wäre es ja zu einfach.

Mehr als `json`, `os`, `pathlib`, `fcntl` und `hashlib` aus der Standardbibliothek brauchen Sie dafür nicht.

Womit Sie testen

Ihre API ist ein Programm für Programme – prüfen Sie sie deshalb über HTTP und nicht im Browser. Drei Werkzeuge, in dieser Reihenfolge nützlich:

- **curl auf der Kommandozeile.** Das Arbeitspferd. `curl-beispiele.sh` in diesem Ordner erklärt die nötigen Schalter an acht Beispielen: Header ansehen, Header mitschicken, JSON posten, einen `ETag` aus der Antwort in den nächsten Request übernehmen, zwei Requests gleichzeitig abschicken. Lesen Sie die Datei einmal durch, dann kopieren Sie daraus.
- **Ein Klick-Client.** `requests.http` ist eine Sammlung derselben Anfragen für die Erweiterung *REST Client* (Klick auf „Send Request“ über jedem `###`). *Thunder Client* und *Postman* sind auch installiert, falls Sie eine Oberfläche mit Historie bevorzugen; beide können die Datei importieren.
- **Automatisierte Tests.** Sobald Sie dieselben Prüfungen zum dritten Mal von Hand machen, lohnt `pytest`. Zwei Hinweise: Setzen Sie `DATA_DIR` im Test auf ein leeres temporäres Verzeichnis, damit Tests sich nicht gegenseitig die Ablage verändern. Und die Nebenläufigkeit aus Teil D lässt sich nur gegen einen **echten laufenden Server** mit `--workers 4` prüfen, nicht mit dem `TestClient` im selben Prozess – der arbeitet Requests hintereinander ab.

Was Sie mit keinem dieser Werkzeuge prüfen können, ist Teil G: Ein Absturz mitten im Schreiben lässt sich nicht von außen auslösen. Dafür sehen Sie in die Ablage unter `$DATA_DIR` und in den Prozess selbst.

Teil A – Entwurf

Bevor Sie eine Zeile Code schreiben, entwerfen Sie die API.

A1 – Ressourcen. Legen Sie fest, welche Ressourcen es gibt und unter welchen URIs sie liegen. Mindestens: Pakete, Zustandsänderungen, Zustellungen, Zahlungen. Halten Sie das in einer Tabelle fest:

URI	Methode	Zweck	Erfolg	mögliche Fehler
<code>/parcels</code>	<code>POST</code>	Paket aufgeben	<code>201</code> + <code>Location</code>	<code>400</code> , <code>422</code>
...	...	...	...	...

A2 – Sicher, idempotent, keins von beidem. Markieren Sie in der Tabelle für jede Zeile, ob die Operation *safe* ist (verändert nichts) und ob sie *idempotent* ist (zweimal ausführen hat denselben Effekt wie einmal). Bei jeder Zeile, die weder *safe* noch *idempotent* ist, schreiben Sie einen Satz dazu, was passiert, wenn der Client sie versehentlich zweimal schickt.

A3 – Die Entwurfsentscheidung. „Ein Paket zustellen“ – ist das eine Aktion auf dem Paket oder eine eigene Ressource?

Entscheiden Sie sich, und begründen Sie die Entscheidung. Gehen Sie dabei auf den wiederholten Request ein: Was passiert bei Ihrer Variante, wenn derselbe Request zweimal ankommt? Es gibt hier keine eindeutig richtige Antwort – aber es gibt unbegründete Antworten, und die zählen nicht.

A4 – Fehlerkatalog. Ordnen Sie jeder Fehlersituation einen Statuscode zu und begründen Sie kurz, warum genau dieser. Mindestens diese Situationen:

- Paket erfolgreich angelegt
- Paket-ID existiert nicht
- Pflichtfeld fehlt im Request-Body
- Zustandsübergang fachlich unzulässig (`DELIVERED` → `IN_TRANSIT`)
- Ressource wurde zwischenzeitlich von jemand anderem geändert
- derselbe Request kommt ein zweites Mal an
- die Dateiablage ist nicht schreibbar

A5 – Fehlerformat. Legen Sie *ein* Format für Fehlerantworten fest und halten Sie sich durchgängig daran. Orientieren Sie sich an RFC 9457 (*Problem Details for HTTP APIs*):

```
{
  "type": "https://example.com/probleme/unzulaessiger-uebergang",
  "title": "Unzulässiger Zustandsübergang",
  "status": 409,
  "detail": "Paket 4711 ist DELIVERED und kann nicht IN_TRANSIT werden."
}
```

Teil B – Grundgerüst

Legen Sie Ihr Python-Projekt an (`uv init` , siehe `README.md`) und implementieren Sie die Basis.

B1 – Die Ablage. Nutzen Sie diese Verzeichnisstruktur unter `$DATA_DIR` :

```
$DATA_DIR/
├─ parcels/<id>.json      eine Datei pro Paket
├─ tracking/<id>.log      Zustandsänderungen, eine Zeile je Ereignis, nur
angehängt
...
```

Welche Felder in den JSON-Dateien stehen, entscheiden Sie.

B2 Trennen Sie den Zugriff auf die Files von den HTTP-Handlern, etwa in einem Modul `store.py` .

B3 Pakete anlegen und lesen. `POST` antwortet mit `201` , einem `Location` -Header auf die neue Ressource und der Repräsentation im Body.

B4 Eine Liste der Pakete mit Filter nach Zustand, etwa `GET /parcels?status=IN_TRANSIT` .

B5 Sinnvolle Validierung. Ein Request ohne Empfängeradresse oder ein Paket mit negativem Gewicht darf nicht mit `500` beantwortet werden.

Teil C – Der Zustandsautomat

C1 Implementieren Sie die Zustandsübergänge aus dem Diagramm.

C2 Ein unzulässiger Übergang wird abgelehnt, ohne den Zustand zu ändern. Statuscode: der aus A4. Die Antwort nennt den aktuellen und den gewünschten Zustand.

C3 Jede Zustandsänderung wird mit Zeitstempel an `tracking/<id>.log` angehängt, `GET /parcels/{id}/tracking` liefert sie chronologisch.

Teil D – Zwei Fahrer, ein Paket

D1 Zwei Fahrer stehen im Depot. Beide laden dasselbe Paket auf ihr Terminal, beidemmarkieren es als übernommen (`assigned_driver`). Ohne Gegenmaßnahme gewinnt der letzte Schreibende, und das Paket liegt in zwei Touren – *lost update*.

Wie könne Sie ein solches Szenario testen?

Teil E – Ein Request, der zweimal ankommt

Ein Handterminal bucht die Versandkosten. Die Zahlung geht durch, die Antwort geht auf dem Rückweg verloren. Das Terminal sieht nur einen Timeout und schickt den Request erneut. Ohne Gegenmaßnahme zahlt der Kunde zweimal.

Der entscheidende Satz dazu: **Ein Netzwerkfehler ist kein Beweis dafür, dass die Operation nicht ausgeführt wurde.** Der Client *kann* nicht wissen, was passiert ist. Also muss der Server das Problem lösen.

Client-Fehler: Derselbe Schlüssel kommt mit einem *anderen* Body an (erst 12,90 €, dann 99,00 €). Das ist ein Client-Fehler, keine Wiederholung. Wählen Sie einen Statuscode und dokumentieren Sie ihn. Tipp: Ein Hash über den Request-Body, neben der Antwort gespeichert, macht die Erkennung billig.

Teil F – Caching

Die Sendungsverfolgung wird von Kunden im Minutentakt aufgerufen. Jeder Aufruf liest Ihre Dateien – muss er aber nicht.

Entscheiden Sie pro Ressource, ob sie überhaupt cachebar ist. Die Zahlungshistorie eines Kunden gehört vermutlich nicht in einen gemeinsam genutzten Cache – welcher `Cache-Control` -Wert drückt das aus?

Wählen Sie eine `max-age` -Zeit für relevanten Ressourcen und begründen Sie sie. Z.B: Wie veraltet darf eine Sendungsverfolgung sein, bevor der Kunde sich beschwert?

Werten Sie `If-None-Match` aus. Passt der Wert, antworten Sie mit `304` und **ohne Body**.

Teil G – Störungen

Gehen Sie folgenden Szenarien gedanklich durch. Was haben passiert, was würde die API antworten, wie sah die File-DB danach aus, und was ist das gewünschte Verhalten?

1. **Doppelter Request.** Denselben `POST` zweimal absetzen – einmal mit, einmal ohne `Idempotency-Key` . Wie viele Datensätze entstehen jeweils?
 2. **Verlorene Antwort.** Den Client mitten in der Antwort abbrechen und den Request wiederholen. Der Server hat die Operation ausgeführt – merkt der Client das?
 3. **Gleichzeitige Änderung.** Zwei `PUT` mit demselben `If-Match` parallel (Teil D). Nur ein `PUT` darf gewinnen.
 4. **Absturz beim Schreiben.** Die Daten wurden nicht geschrieben, der Client hat nie eine Antwort bekommen. Was macht der Client beim nächsten Versuch – und was antwortet Ihre API?
 5. **Absturz nach dem Schreiben, vor der Antwort.** `os._exit(1)` direkt nach dem erfolgreichen `os.replace` . Die Daten sind da, der Client hat nie eine Antwort bekommen. Was macht der Client beim nächsten Versuch – und was antwortet Ihre API?
 6. **Veralteter Cache.** Ein Paket ändern, während die alte Antwort noch innerhalb der `max-age` -Zeit liegt. Wie lange sieht ein Client die alte Information? Ist das vertretbar?
 7. **Ablage nicht schreibbar.** Simulieren Sie eine vollgelaufene oder schreibgeschützte Platte.
-

Kür

Freiwillig, wenn Sie früh fertig sind:

- **HATEOAS**. Legen Sie `_links` in die Repräsentation – aber nur die Übergänge, die im aktuellen Zustand *erlaubt* sind. Ein `DELIVERED` -Paket hat keinen Link `deliver`. Damit wird der Zustandsautomat für den Client sichtbar.
 - **429 Too Many Requests** mit `Retry-After` für die Sendungsverfolgung.
 - **Paginierung** für `GET /parcels` – und die Frage, was passiert, wenn zwischen Seite 1 und Seite 2 ein Paket dazukommt.
 - **DELETE** – wie machen Sie es idempotent? Was antwortet der zweite Aufruf?
 - **Write-Ahead-Log**. Sie haben mit `tracking/<id>.log` schon eins. Was wäre, wenn der Paketzustand *nur* aus diesem Log rekonstruiert würde und `parcels/<id>.json` bloß ein Zwischenspeicher wäre? Das ist die Grundidee von Event Sourcing – und die Antwort vieler verteilter Systeme auf Teil G.4.
 - **Ein Testfall pro Störung** aus Teil G, automatisiert mit `pytest`.
-